

1BM24CS299
SUMANT SHRIDHAR
5K BATCH 2

AI LAB 2

9/10/26

1. 8 PUZZLE USING DFS

CODE:

```
def dfs(state, goal, path, visited):

    if state == goal:
        return path

    visited.add(state)

    # Find blank position
    blank = state.index(0)
    row = blank // 3
    col = blank % 3

    # Possible moves: Up, Down, Left, Right
    # moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    moves = [(0, 1), (1, 0), (-1, 0), (0, -1)]

    for move in moves:

        new_row = row + move[0]
        new_col = col + move[1]

        if 0 <= new_row < 3 and 0 <= new_col < 3:

            new_blank = new_row * 3 + new_col

            new_state = list(state)

            # Move tile into blank space
            new_state[blank] = new_state[new_blank]
            new_state[new_blank] = 0

            new_state = tuple(new_state)

            if new_state not in visited:

                result = dfs(new_state, goal, path + [new_state], visited)

                if result is not None:
                    return result

    return None

# Initial and goal states
initial = (1, 2, 3,
           4, 5, 6,
           0, 7, 8)

goal = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)
```

```

visited = set()

solution = dfs(initial, goal, [initial], visited)

if solution:
    print("Solution found!")

    for state in solution:
        print()
        print(state[0], state[1], state[2])
        print(state[3], state[4], state[5])
        print(state[6], state[7], state[8])
else:
    print("No solution found.")

```

OUTPUT:

Solution found!

```

1 2 3
4 5 6
0 7 8

```

```

1 2 3
4 5 6
7 0 8

```

```

1 2 3
4 5 6
7 8 0

```

2. 8 PUZZLE USING IDS

CODE:

```
def dfs(state, goal, depth, path, visited):

    if state == goal:
        return path

    if depth == 0:
        return None

    visited.add(state)

    # Find blank position
    blank = state.index(0)
    row = blank // 3
    col = blank % 3

    # Up, Down, Left, Right
    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for move in moves:

        new_row = row + move[0]
        new_col = col + move[1]

        if 0 <= new_row < 3 and 0 <= new_col < 3:

            new_blank = new_row * 3 + new_col

            new_state = list(state)

            new_state[blank] = new_state[new_blank]
            new_state[new_blank] = 0

            new_state = tuple(new_state)

            if new_state not in visited:

                result = dfs(new_state, goal, depth - 1,
                             path + [new_state], visited)

                if result is not None:
                    return result

    return None


def ids(initial, goal):

    depth = 0

    while True:

        visited = set()

        solution = dfs(initial, goal, depth, [initial], visited)
```

```

        if solution is not None:
            return solution

        depth = depth + 1

# Initial state
initial = (1, 2, 3,
          0, 4, 6,
          7, 5, 8)

# Goal state
goal = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)

solution = ids(initial, goal)

print("Solution found!")

for state in solution:
    print()
    print(state[0], state[1], state[2])
    print(state[3], state[4], state[5])
    print(state[6], state[7], state[8])

```

OUTPUT:

```

... Solution found!

```

```

1 2 3
0 4 6
7 5 8

```

```

1 2 3
4 0 6
7 5 8

```

```

1 2 3
4 5 6
7 0 8

```

```

1 2 3
4 5 6
7 8 0

```